Instantly share code, notes, and snippets.

FiloSottile / **32.asm**

Last active 6 days ago

Embed <script src="https://gi Download ZIP

NASM Hello World for x86 and x86_64 Intel Mac OS X (get yourself an updated nasm with brew)

32.asm

```
; /usr/local/bin/nasm -f macho 32.asm && ld -macosx_version_min 10.7.0 -o 32 32.o && ./32

global start

section .text
start:
    push    dword msg.len
    push    dword msg
    push    dword 1
    mov     eax, 4
    sub     esp, 4
    int     0x80
    add     esp, 16

    push    dword 0
    mov     eax, 1
    sub     esp, 12
    int     0x80

section .data

msg:    db      "Hello, world!", 10
.len:   equ     $ - msg
```

64.asm

```
; /usr/local/bin/nasm -f macho64 64.asm && ld -macosx_version_min 10.7.0 -lSystem -o 64 64.o && ./64

global start


section .text

start:
    mov     rax, 0x2000004 ; write
    mov     rdi, 1 ; stdout
    mov     rsi, msg
    mov     rdx, msg.len
    syscall

    mov     rax, 0x2000001 ; exit
    mov     rdi, 0
    syscall


section .data
```

```

msg:    db      "Hello, world!", 10
.len:   equ     $ - msg
```

References.markdown

- NASM Manual - for the syntax
- What are the calling conventions for UNIX & Linux system calls on x86-64 - comparison
- **AMD64 ABI reference - A.2.1** - for the actual x86_64 calling convention
- X86 calling conventions - moar calling conventions
- **Mac OS X syscalls reference**

bannychen commented on May 12, 2014

hi, i run "nasm -f macho64 64.asm" ok,
but run "ld -macosx_version_min 10.7.0 -lSystem -o 64 64.o" result is "-bash: ld: command not found",please help me
what should i do?
many Thanks!

metacritical commented on Sep 14, 2014

You dont have gcc installed just install gcc, actually on mac you need to just install developer tools that come with xcode.

benwei commented on Feb 22, 2015

if you already installed xcode, you can use "xcode-select --install" to install command line developer tools. have been tested on osx 10.9.5

LionelZoc commented on Mar 19, 2015

hi !, instead of :" mov rsi, msg", use: "lea rsi, [rel msg]"

it should work

banxi1988 commented on Aug 17, 2015

@LionelZoc Thanks It Works

dpwell commented on Aug 22, 2015

Hi

when I tried nasm -f macho64 64.asm I got
nasm: fatal: unrecognised output format `macho64'
I tried to find "nasm with brew" as suggested above but couldn't find it. (this is on 10.10.5)

thanks.

deadalnix commented on Aug 27, 2015

@dpwell , if you have a solution, I'm taking.

iCurlmyster commented on Sep 2, 2015

@dpwell nasm should be downloadable through brew. because when i do 'brew search nasm' it comes back with nasm. but if you can't find it through brew just download the newest version from their website because I had the same problem you had and the way I fixed it was just updating nasm to the newest version. But I did mine through brew.

eulerworks commented on Oct 4, 2015

(change your program name to suit, but this worked on 10.10.4 with nasm from brew)

nasm -fmacho simple.s && ld -macosx_version_min 10.7.0 -o simple simple.o && ./simple

uname -a
14.4.0 Darwin Kernel Version 14.4.0: Thu May 28 11:35:04 PDT 2015; root:xnu-2782.30.5~1/RELEASE_X86_64 x86_64

gcc --version
gcc (GCC) 4.9.2 20141029 (prerelease)
Copyright (C) 2014 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

adriansr commented on Oct 9, 2015

If someone is having the "fatal: No section for index 2 offset 0 found" error even after replacing the mov with a lea instruction, this is a problem of recent (2.11.x) nasm available from brew. Works fine with nasm 2.10 from nasm's website.

MareoRaft commented on Dec 18, 2015

I'm on Mavericks 10.9.5. I'm using nasm 2.11.08_1 from brew. I have gcc:

Configured with: --prefix=/Applications/Xcode.app/Contents/Developer/usr --with-gxx-include-dir=/usr/include/c++/4.2.1
Apple LLVM version 6.0 (clang-600.0.56) (based on LLVM 3.5svn)
Target: x86_64-apple-darwin13.4.0
Thread model: posix

I got

```
% nasm -f macho world.asm
world.asm:9: error: symbol `rax' undefined
world.asm:10: error: symbol `rdi' undefined
world.asm:11: error: symbol `rsi' undefined
world.asm:12: error: symbol `rdx' undefined
world.asm:15: error: symbol `rax' undefined
world.asm:16: error: symbol `rdi' undefined
```

but fixed it by changing the "r"s to "e"s in the file. Then it compiled with

```
% ld -macosx_version_min 10.7.0 -lSystem -o world world.o
```

successfully. But when I execute it, I get

```
% ./world
zsh: illegal hardware instruction  ./world
```

D3m0t3p commented on Mar 9, 2016

i have the same problem as you **@MareoRaft**

Leandros commented on Apr 23, 2016

@MareoRaft @D3m0t3p You're using the 64-Bit assembly, but compile it as 32-Bit. It's obvious that this doesn't work.

ourui commented on May 25, 2016

@dpwell I install nasm from brew, and the same problem with you. When I type 'nasm -v' , the result is 'NASM version 0.98.40 (Apple Computer, Inc. build 11) compiled on Feb 10 2016' . I doubt that the terminal has use system nasm. So, I close the terminal and restart, and it works. Now nasm can recognised 'macho64'. Type 'nasm -v' again, the result is 'NASM version 2.12.01 compiled on Mar 23 2016'.

tilarids commented on Sep 22, 2016

I've done a 32-bit version that works on 10.11.5: https://gist.github.com/tilarids/3fe6d8499ad25b722c307cfcfb817dc2
It seems padding and symbol table are now necessary.

lwickstr commented on Oct 23, 2016

for those having trouble with the "install nasm from brew" directions above, here is a guide to install brew:
http://www.howtogeek.com/211541/homebrew-for-os-x-easily-installs-desktop-apps-and-terminal-utilities/
(i only needed the one line command:
ruby -e "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/master/install)"
)
then running:
nasm install brew
will give you nasm version 2.12+

shoaibmohi commented on May 24, 2017

How to run NASM code in Xcode on MacOS Sierra.
Please Help.

sguillia commented on Sep 10, 2017

When I run the program, the return value is 1, but if I change `sub esp, 12` to `sub esp, 4` as it was in old revisions, then it works, return value 0. Why does it happen like that ? On OS X Sierra 10.12.6

kwccoin commented on Oct 15, 2017 • edited

I also need to do :

```
push   dword 0
mov    eax,  1  ; 0 ; should not be 1
sub    esp,  4  ; 12 ; not sure about this and note there is a push dword 0 here
int    0x80
```

to work under OS X Sierra 10.12.6.

Also, the first few line is mysterious

```
section .text
start:
push    dword msg.len ;4 bytes
push    dword msg     ;4 bytes
push    dword 1       ;4 bytes. and there is push dword 1 here
mov     eax, 4
sub     esp, 4        ;reserves 4 bytes more
int     0x80
add     esp, 16       ;restore esp to original value

; usually should be
; push ebp
; mov  ebp, esp
; push ??? the 4 bytes not sure what that for
; ...
; pop  ??? the 4 bytes
; pop  ebp
```

May be c calling standard, Win32 calling standard or ...

HGarron commented on Dec 8, 2017

FiloSottile:

Thank you very much. I am trying to learn assembly language on a Mac.

2019osbjoh commented on Feb 25

When attempting to link I got this:

Undefined symbols for architecture x86_64:
"_main", referenced from:
implicit entry/start for main executable
ld: symbol(s) not found for inferred architecture x86_64

anybody able to shed a little light?

NovelCorpse commented on Mar 4 • edited

Hello, **@2019osbjoh** !
I have replaced `start` to `_main` and it works for me. *(osx version: 10.12; nasm version: 2.14.02)*
src:

```
global _main
section .text
_main:
    mov     rax, 0x2000004
    mov     rdi, 1
    mov     rsi, msg
    mov     rdx, msg.len
    syscall
    mov     rax, 0x2000001
    mov     rdi, 0
    syscall
section .data
msg:    db      "Hello, world!", 10
.len:   equ     $ - msg
```

build:

```
$ nasm -f macho64 hello-world.asm && ld -macosx_version_min 10.12 -lSystem -o hello-world hello-world.o
```

lancejpollard commented on Mar 21 • edited

You can make a custom global start/_main to name it however you want with this:

```
ld -e start -macosx_version_min 10.13.0 -static -o hello-world hello-world.o
```

Using "static" linking instead of dynamic, in which case you can't control the name.